ЛАБОРАТОРНА РОБОТА №1

Тема: Вступ до ASP.NET Core

Мета: ознайомитися з основними принципами роботи .NET, навчитися налаштовувати середовище розробки та встановлювати необхідні компоненти, набути навичок створення рішень та проектів різних типів, набути навичок обробки запитів з використанням middleware.

Хід роботи:

Завдання 2. Створення проєктів.

Частина 1.

1. Створіть проєкт для консольного додатку з назвою ConsoleToWeb з використанням dotnet CLI

```
E:\Вибіркова - ASP.NET\Lab1>dotnet new console -n ConsoleToWeb
The template "Console App" was created successfully.

Processing post-creation actions...
Restoring E:\Вибіркова - ASP.NET\Lab1\ConsoleToWeb\ConsoleToWeb.csproj:
Restore succeeded.



E:\Вибіркова - ASP.NET\Lab1>dotnet add package Microsoft.AspNetCore.App
Could not find any project in `E:\Вибіркова - ASP.NET\Lab1\`.

E:\Вибіркова - ASP.NET\Lab1>cd ConsoleToWeb

E:\Вибіркова - ASP.NET\Lab1\ConsoleToWeb>dotnet add package Microsoft.AspNetCore.App

Build succeeded in 1,4s
info : X.509 certificate chain validation will use the default trust store selected b
y .NET for code signing.
info : X.509 certificate chain validation will use the default trust store selected b
y .NET for timestamping.
info : Adding PackageReference for package 'Microsoft.AspNetCore.App' into project 'E
:\Вибіркова - ASP.NET\Lab1\ConsoleToWeb\ConsoleToWeb.csproj'.
info :   CACHE https://api.nuget.org/v3/registration5-gz-semver2/microsoft.aspnetcore
.app/index.json
info : Restoring packages for E:\Вибіркова - ASP.NET\Lab1\ConsoleToWeb\ConsoleToWeb.c
sproj...
```

Рис. 1. Створення консольного додатку ConsoleToWeb.

2. Перетворіть створений консольний додаток на веб-додаток
3. Опишіть виконані кроки у звіті

Консольний додаток створено за допомогою команди dotnet new console. Далі додано пакет Microsoft.AspNetCore.App і змінено код для обробки HTTP-запитів. У файлі проєкту SDK змінено на Microsoft.NET.Sdk.Web, що перетворило проєкт на веб-додаток.

Лістинг Program.cs:

```
using Microsoft.AspNetCore.Builder;
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Http;

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.MapGet("/", async context =>
{
    await context.Response.WriteAsync("Hello from ConsoleToWeb!");
});

app.Run();
```

Лістинг ConsoleToWeb.csproj:

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net9.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.AspNetCore.App" Version="2.2.8" />
  </ItemGroup>

</Project>
```

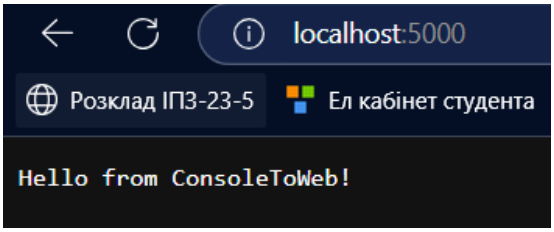

Рис. 2. Результат виконання.

Частина 2.

1. Створіть ASP.NET WebAPI проєкт без авторизації з назвою WebFromCli з використанням dotnet CLI

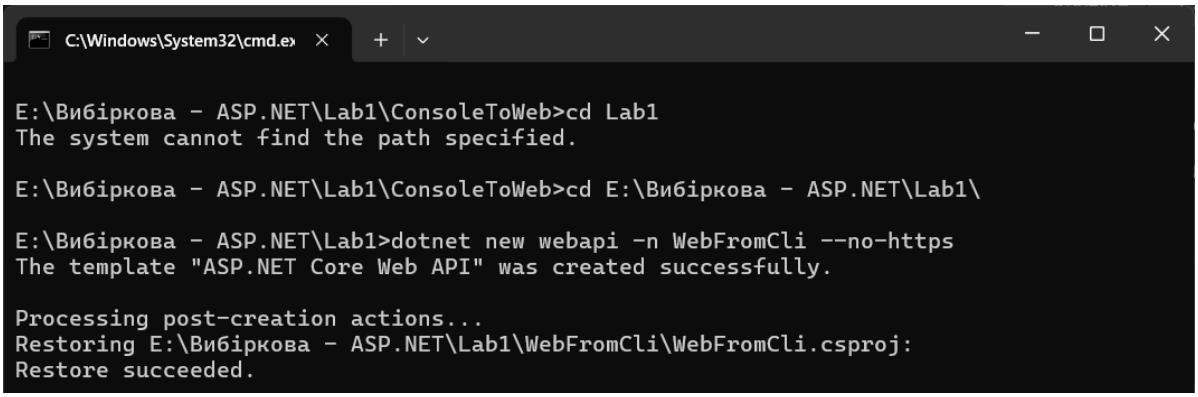

Рис. 3. Створення проєкту WebFromCli.

2. Реалізуйте GET-ендпоінт “/who”, який повертатиме у відповідь ваше ім’я та прізвище
3. Реалізуйте GET-ендпоінт “/time”, який повертатиме у відповідь поточний час на сервері

4. Наведіть лістинг реалізованих обробників у звіті

Лістинг Program.cs:

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
// Learn more about configuring OpenAPI at https://aka.ms/aspnet/openapi
builder.Services.AddOpenApi();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
}

var summaries = new[]
{
    "Freezing", "Bracing", "Chilly", "Cool", "Mild", "Warm", "Balmy", "Hot",
"Sweltering", "Scorching"
};
app.MapGet("/", () => "Welcome to WebFromCli!");
app.MapGet("/who", () => "Ліна Бубон");
app.MapGet("/time", () => DateTime.Now.ToString("HH:mm:ss"));
app.MapGet("/weatherforecast", () =>
{
    var forecast =  Enumerable.Range(1, 5).Select(index =>
        new WeatherForecast
        (
            DateOnly.FromDateTime(DateTime.Now.AddDays(index)),
            Random.Shared.Next(-20, 55),
            summaries[Random.Shared.Next(summaries.Length)]
        ))
        .ToArray();
    return forecast;
})
.WithName("GetWeatherForecast");

app.Run();

record WeatherForecast(DateOnly Date, int TemperatureC, string? Summary)
{
    public int TemperatureF => 32 + (int)(TemperatureC / 0.5556);
}
```

Створено WebAPI проєкт без авторизації за допомогою dotnet new webapi.

Реалізовано два ендпоінти: /who повертає ім’я та прізвище, а /time -поточний час на сервері у форматі "HH:mm:ss".

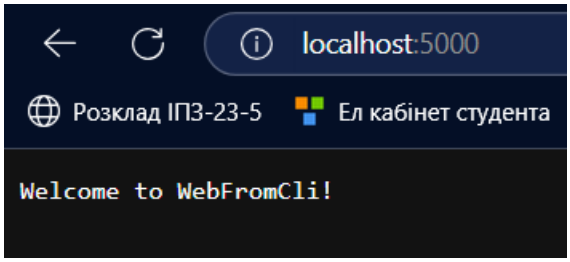

Рис. 4. Результат виконання /.

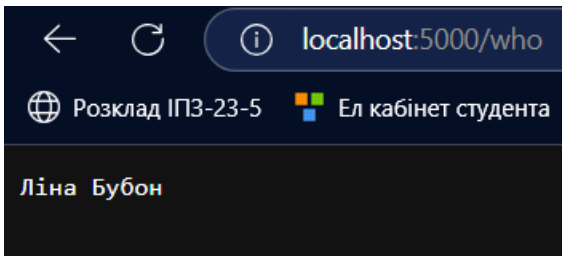

Рис. 5. Результат виконання /who.

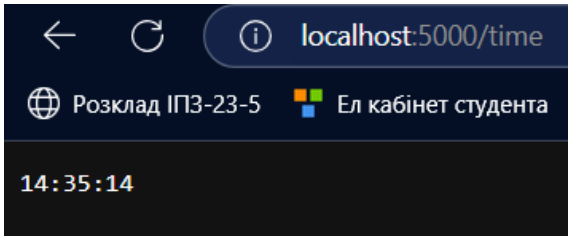

Рис. 6. Результат виконання /time.

Частина 3.

1. Створіть ASP.NET MVC проєкт будь-яким зручним способом.

```
C:\Windows\System32\cmd.e›

E:\Вибіркова - ASP.NET\Lab1\WebFromCli>cd E:\Вибіркова - ASP.NET\Lab1\

E:\Вибіркова - ASP.NET\Lab1>dotnet new mvc -n MvcLab
The template "ASP.NET Core Web App (Model-View-Controller)" was created successfully.
This template contains technologies from parties other than Microsoft, see https://ak
a.ms/aspnetcore/9.0-third-party-notices for details.

Processing post-creation actions...
Restoring E:\Вибіркова - ASP.NET\Lab1\MvcLab\MvcLab.csproj:
Restore succeeded.
```

Рис. 7. Створення проєкту MvcLab.

2. Реалізуйте контролер з назвою LabController
3. В створеному контролері реалізуйте обробник /info, який повертатиме View з даними про номер лабораторної роботи, тему, мету та ім’я та прізвище виконавця в табличному вигляді.

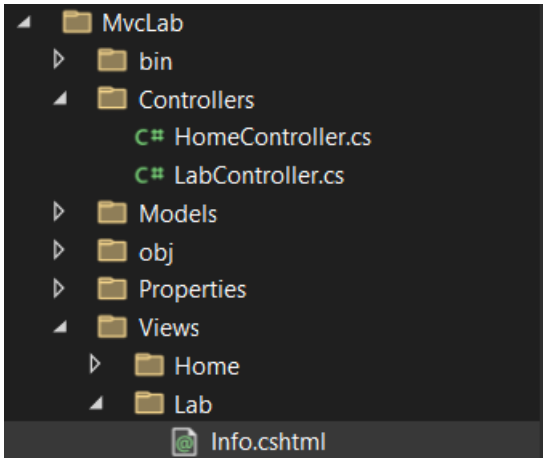

Рис. 8. Структура проєкту з контролером та Info.cshtml зі шляхом Views/Lab.

Лістинг LabController.cs:

```
using Microsoft.AspNetCore.Mvc;

namespace MvcLab.Controllers
{
	public class LabController : Controller
	{
		public IActionResult Info()
		{
			ViewBag.LabNumber = "1";
			ViewBag.Topic = "Створення проєктів";
			ViewBag.Purpose = "ознайомитися з основними принципами роботи .NET, навчитися налаштовувати середовище розробки та встановлювати необхідні компоненти, набути навичок створення рішень та проектів різних типів, набути навичок обробки запитів з використанням middleware.";
			ViewBag.Author = "Ліна Бубон";
			return View();
		}
	}
}
```

Лістинг Info.cshtml:

```
<table border="1">
<tr>
<th>Номер лабораторної</th>
<td>@ViewBag.LabNumber</td>
</tr>
<tr>
<th>Тема</th>
<td>@ViewBag.Topic</td>
</tr>
<tr>
<th>Мета</th>
<td>@ViewBag.Purpose</td>
</tr>
<tr>
<th>Авторка</th>
<td>@ViewBag.Author</td>
</tr>
</table>
```

4. Дані для відображення передати з контролера

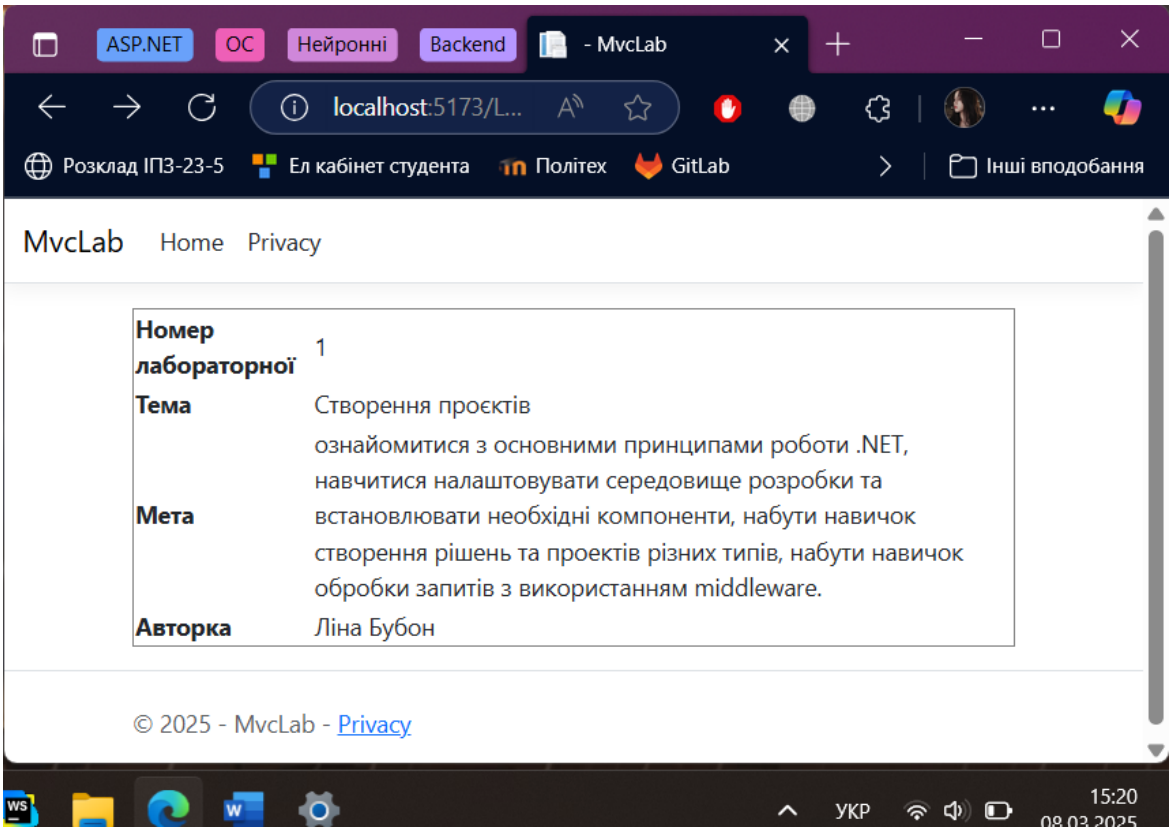

Рис. 9. Результат виконання.

Створено MVC проєкт через dotnet new mvc. Додано контролер LabController з методом Info, який передає дані через ViewBag. У представленні Info.cshtml дані відображено в таблиці.

Завдання 3. Робота з middleware

1. Ознайомтеся з поняттями middleware в ASP.NET
2. Створіть ASP.NET WebAPI проєкт з назвою MiddlewareSandbox

```
E:\Вибіркова - ASP.NET\Lab1>dotnet new webapi -n MiddlewareSandbox --no-https
The template "ASP.NET Core Web API" was created successfully.

Processing post-creation actions...
Restoring E:\Вибіркова - ASP.NET\Lab1\MiddlewareSandbox\MiddlewareSandbox.csproj:
Restore succeeded.
```

Рис. 10. Створення проєкту MiddlewareSandbox.

3. Реалізуйте наступні middleware

   а. Реалізуйте middleware, яке рахує кількість запитів до сервера і повертає це число у відповіді, наприклад: The amount of processed requests is X.

b. Створіть middleware, яке аналізує параметри рядка запиту. Якщо в рядку запиту (query string) присутній параметр “custom”, то повертати у відповідь “You’ve hit a custom middleware!”, інакше пропускати запит далі.

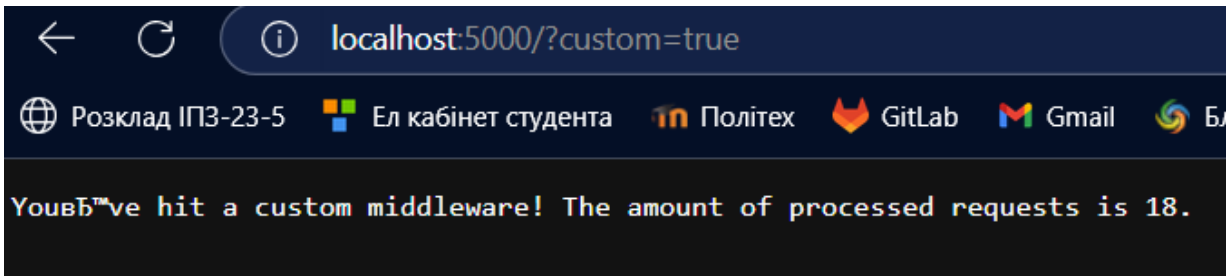

Рис. 11. Результат виконання підрахунку кількості запитів та аналізу рядка запиту.

c. Створіть middleware, яке логуватиме у консоль інформацію про всі запити. В логах відображати метод запиту (GET, POST тощо) та його шлях (/user, /product тощо). Для перевірки POST та інших методів запиту можна скористатися Postman або іншою подібною утилітою.

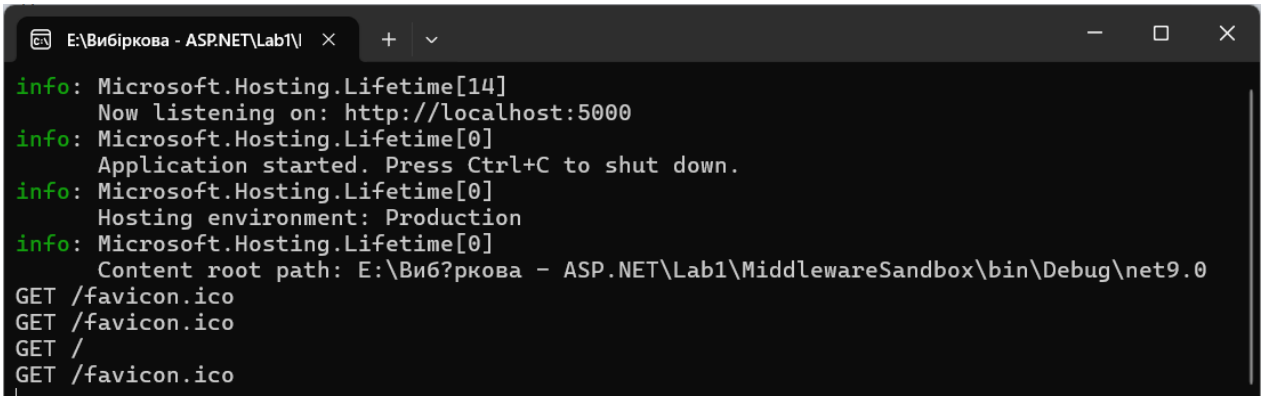

Рис. 12. Вивід у консоль методів (GET) і шляхів (/ та /favicon.ico).

d. Реалізуйте middleware, яке перевіряє наявність API-ключа у заголовках запиту. Для цього слід Перевіряти, чи є в запиті заголовок X-API-KEY. Якщо ключ неправильний або відсутній, повертати 403 Forbidden, не передавати запит для виконання далі. Якщо ключ в заголовку запиту співпадає з тим, який заданий на сервері, передавати запит далі.

Рис. 13. Перевірка API-ключа.

Лістинг Program.cs:

```
uilder = WebApplication.CreateBuilder(args);
var app = builder.Build();

int requestCount = 0;
app.Use(async (context, next) =>
{
    requestCount++;
    await next(context);
    if (!context.Response.HasStarted)
    {
        await context.Response.WriteAsync($"The amount of processed requests is
{requestCount}.\n");
    }
});

app.Use(async (context, next) =>
{
    if (context.Request.Query.ContainsKey("custom"))
    {
        await context.Response.WriteAsync("You've hit a custom middleware!");
        return;
    }
    await next(context);
});

app.Use(async (context, next) =>
{
    Console.WriteLine($"{context.Request.Method} {context.Request.Path}");
    await next(context);
});

app.Use(async (context, next) =>
{
    const string validApiKey = "secret-key";
    if (!context.Request.Headers.TryGetValue("X-API-KEY", out var apiKey) || apiKey !=
validApiKey)
    {
        context.Response.StatusCode = 403;
        await context.Response.WriteAsync("Invalid or missing API key.");
        return;
    }
    await next(context);
});

app.MapGet("/", () => "Welcome to MiddlewareSandbox!");

app.Run();
```

Створено:

Лічильник запитів, що рахує кількість запитів і додає повідомлення до відповіді після обробки, якщо відповідь ще не почалася.

Аналіз параметра "custom",що перевіряє наявність параметра "custom" у рядку запиту та повертає повідомлення, зупиняючи обробку.

Логування запитів, котрий иводить у консоль метод і шлях кожного запиту (наприклад, "GET /").

Перевірка API-ключа, котра перевіряє заголовок X-API-KEY. Якщо ключ неправильний або відсутній, повертає 403 Forbidden і зупиняє обробку.

Висновок: в ході лабораторної роботи №1 було ознайомлено з основними принципами роботи .NET, навчено на практиці налаштовувати середовище розробки та встановлювати необхідні компоненти, були набуті навички створення рішень та проєктів різних типів, також навички обробки запитів з використанням middleware.

Посилання на репозиторій:
IPZ / 2023-2027 / IPZ-23-5 / Bubon Lina / ASP.NET · GitLab